

Salary Management System

Backend Approach & Feature Overview — aligned with the assessment requirements

1. Backend Approach

FastAPI + SQLAlchemy + PostgreSQL API designed for HR to manage approximately 10,000 multi-country employees and their compensation data. The backend follows a **layered architecture**: **API** → **Services** → **Repositories** → **Models**, keeping HTTP handling, business rules, database access, and persistence concerns separated.

This approach aligns with the assessment's emphasis on maintainable structure, relational database usage, realistic 10,000-employee seed data, meaningful tests, performance awareness, and a functional HR-facing product.

2. Problems Solved

Business problem	Backend solution
HR cannot efficiently manage thousands of employees in spreadsheets	Paginated, searchable, filterable and sortable employee CRUD APIs
Pay differs by country and currency	Country/currency master data plus Frankfurter FX rates for reporting conversion
Ongoing package pay and period-specific pay can be confused	Salary profiles represent ongoing compensation; salary records represent period pay lines
Compensation contains multiple earnings and deductions	Configurable salary components: earning/deduction with fixed, percentage or per-unit calculation
HR needs active, paid/unpaid and other operational views	Employment, salary-record and payment status exposed through list and analytics filters
Cross-country salary comparison is difficult	Analytics can normalize compensation into a selected reporting currency
Lists alone do not answer organization-wide pay questions	Summary, country, department, level, distribution and trend analytics
An empty database is poor for demos and performance testing	Deterministic seed data for 10,000 employees, salary profiles and records

3. Backend Features

Master Data: Countries, currencies, departments and job levels provide controlled reference data.

Employees: Create, update, list and delete; search by name; filter by department, level, employment status, currency and salary range; pagination and sorting.

Salary Profiles: Ongoing compensation package, salary history, pay frequency from weekly through yearly, and preview-before-save support.

Salary Records: Period-specific pay lines with components, effective dates, ACTIVE/INACTIVE/DRAFT lifecycle and PAID/NOT_PAID status.

Salary Components: Reusable catalog of earnings/deductions such as basic, HRA, overtime, bonus and deductions.

Exchange Rates: External FX rates support conversion into a common reporting currency for lists and analytics.

Analytics: Filtered reporting with summary, country/department/level breakdowns, salary distribution and trends; supports date range or as-of snapshot.

Operations: Health endpoint, friendly error handling and Docker startup behavior that can seed an empty database.

4. Layered Structure

Layer	Responsibility
api/	HTTP routes and request/response handling (/api/v1/...)
services/	Business rules: salary calculation, FX conversion, analytics
repositories/	Database queries: filtering, sorting, pagination, eager loading
models/	SQLAlchemy tables, relationships and enums

Layer	Responsibility
schemas/	Pydantic request/response contracts
seed/	Deterministic demo dataset for realistic testing

5. Core Domain Separation

Employee = who the person is. **Salary Profile** = what the person is paid on an ongoing basis. **Salary Record** = what was paid for a particular period. **Analytics** = organization-wide insights calculated from these source records and normalized through FX.

6. Scale & Engineering Considerations

- Seed 10,000 employees with deterministic data so demos and tests are repeatable.
- Use server-side pagination, filtering and sorting instead of loading the entire employee dataset into the UI.
- Use database indexes for common employee, organization, status and salary queries.
- Keep salary calculation and currency conversion in the service layer rather than in route handlers.
- Use relational constraints and transactional updates for salary-related data integrity.
- Use Decimal-based money calculations and avoid treating floating-point values as the financial source of truth.
- Design analytics to aggregate at the database/service layer rather than transferring all employee rows to the frontend.

7. Assessment Alignment

The backend directly supports the assessment goals: an HR Manager persona, management of salary information for an organization of roughly 10,000 employees, relational persistence, seeded demo data, clean code structure, testability, performance considerations, and analytics that help HR understand how the organization pays its people. The implementation intentionally focuses on salary management and compensation insight rather than becoming a full payroll/HRMS platform.

8. Architecture Flow

React / Next.js UI → **FastAPI API** → **Service Layer** → **Repository Layer** → **SQLAlchemy Models** → **PostgreSQL**
 ■ FX / Analytics services

Note: This document describes the backend approach supplied for this Salary Management System and keeps the terminology of salary profiles and salary records used in the current backend design.